C PROGRAMMING ASSIGNMENT 1

DESIGN, IMPLEMENT AND VALIDATE A RELIABLE BILLING PROGRAM FOR A RURAL ELECTRICITY CO-OPERATIVE

Student Name: Sri Varshini K

Register Number: 192512328

Programming Language: C

Compiler: GCC 6.3.0

Submission Evidence: Tasks 01–12

S.No	CONTENT
1	Introduction and Problem Statement
2	Objectives and Problem Analysis
3	Data Types and Tariff Constants
4	Algorithms and Pseudocode
5	UML / Module Diagrams
6	Implementation Evidence: Tasks 01–12
7	Test Plan and Results
8	Compilation Stages
9	Complexity Analysis
10	Numeric Accuracy and Overflow
11	Smart Meter Design Challenge
12	Limitations and Future Improvements
13	Conclusion
14	References
Appendix A. Screenshot Evidence	
Appendix B. Task-by-Task Alternative Concepts and Complexity Analysis	

1. Introduction and Problem Statement

Electricity billing requires accurate consumption calculations, correct tariff application, and reliable handling of exceptional conditions such as meter rollover and invalid input. This assignment applies C programming concepts to a rural electricity co-operative billing scenario.

Tariff rules used in the assignment:

Component	Rule
Fixed charge	₹50.00 per bill
First 100 units	No energy charge
Units 101–200	₹2.25 per unit
Units above 200	₹4.50 per unit
Meter register	16-bit unsigned range 0–65535; rollover to 0

The task folders contain separate demonstrations rather than one integrated production application. The smart meter design in Task 12 is a proposal; it is not a complete implementation of telemetry, persistence, or runtime tariff loading.

2. Objectives and Problem Analysis

• Calculate electricity charges according to the given slab structure.

• Represent monetary values as integer paise to avoid binary floating-point errors in billing arithmetic.

• Calculate meter consumption and demonstrate rollover handling.

• Demonstrate data types, macros, conditional compilation, input validation, type conversion, and GCC compilation stages.

• Test ordinary, boundary, rollover, and invalid-input scenarios.

• Describe a smart meter design with 15-minute readings and a 20% peak-hour uplift from 6 PM to 10 PM.

2.1 Inputs, Processing, Outputs

Category	Details
Inputs	Previous/current meter readings; or directly entered consumption units; smart-meter proposal additionally uses timestamps and tariff settings.
Processing	Validate input; compute consumption; account for rollover; calculate energy charge by slab; add ₹50 fixed charge.
Outputs	Consumption units, bill in rupees and paise, data-type/precision demonstrations, and error messages for invalid input.

2.2 Alternative Representation

Integer representation permits direct arithmetic and requires O(1) time and O(1) auxiliary space for a single fixed-width reading calculation. String representation requires validation and conversion before arithmetic; conversion takes O(d) time for d digits.

3. Data Types and Tariff Constants

Type / constant	Role in the project	Important note
unsigned short / meter_t	Meter register value	The C standard does not guarantee 16 bits; verify USHRT_MAX == 65535 on the target platform.
unsigned int	Non-negative consumption	Used for consumption output and combined tests.
long / long long	Bill paise and accumulated totals	Choose and validate a type large enough for the maximum supported consumption and totals.
int	Entered units and ordinary integer examples	Negative values must be rejected where units cannot be negative.
float / double	Floating-point demonstration	Not preferred for exact monetary arithmetic.
RATE2_PAISE = 225	₹2.25 per unit	Integer paise.
RATE3_PAISE = 450	₹4.50 per unit	Integer paise.
FIXED_CHARGE_PAISE = 5000	₹50.00 fixed charge	Integer paise.

3.1 Billing Formula

```
If U <= 100: energy_paise = 0
If 100 < U <= 200: energy_paise = (U - 100) * 225
If U > 200: energy_paise = (100 * 225) + ((U - 200) * 450)
total_paise = 5000 + energy_paise
```

The formula above matches the implemented slab calculations. Input and arithmetic range checks are required before using this formula for unrestricted values.

4. Algorithms and Pseudocode

4.1 Meter Consumption and Rollover

```
INPUT previous, current
IF either reading is outside 0..65535:
   REPORT invalid reading
ELSE IF current >= previous:
   units = current - previous
ELSE:
   units = (65536 - previous) + current
OUTPUT units
```

This pseudocode describes a 16-bit register. The implementation must verify that the selected C type is exactly suitable for this range.

4.2 Billing

```
INPUT units
IF input is not a valid whole number OR units < 0:
   REPORT input error
ELSE:
   energy_paise = 0
   IF units > 200:
      energy_paise = 100 * 225 + (units - 200) * 450
   ELSE IF units > 100:
      energy_paise = (units - 100) * 225
   total_paise = 5000 + energy_paise
   DISPLAY total_paise as rupees and two-digit paise
```

4.3 Smart Meter Processing Proposal

```
LOAD and validate tariff configuration
FOR each timestamped reading:
   validate meter id, timestamp, and reading
   identify whether timestamp is in 18:00–22:00 peak window
   select normal tariff or 1.20 × normal tariff
   calculate interval consumption according to the defined meter policy
   accumulate energy charge using integer paise
STORE readings and billing audit data
ADD fixed charge and DISPLAY bill
```

The uploaded Task 12 notes propose this workflow. They do not include a compiled smart-meter implementation or actual runtime configuration parser.

5. UML / Module Diagrams

Figure 1. Simple use-case diagram for the billing application.

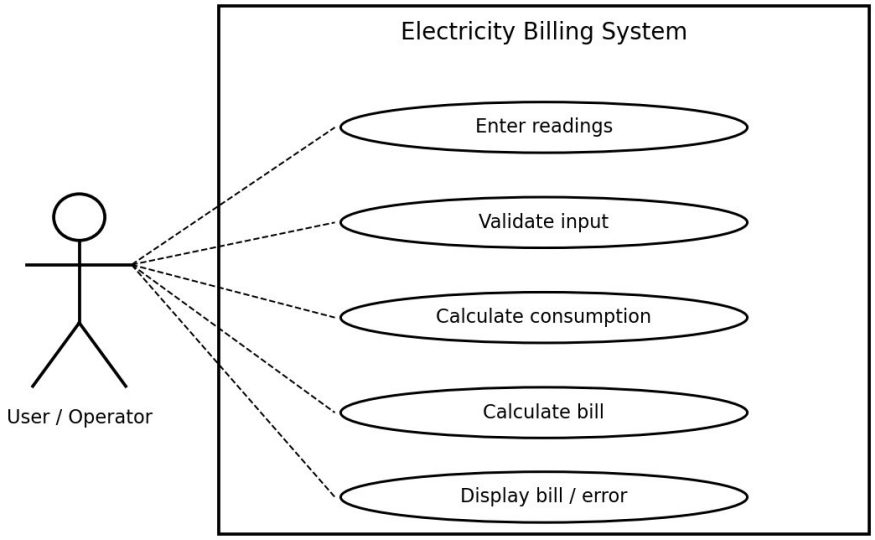
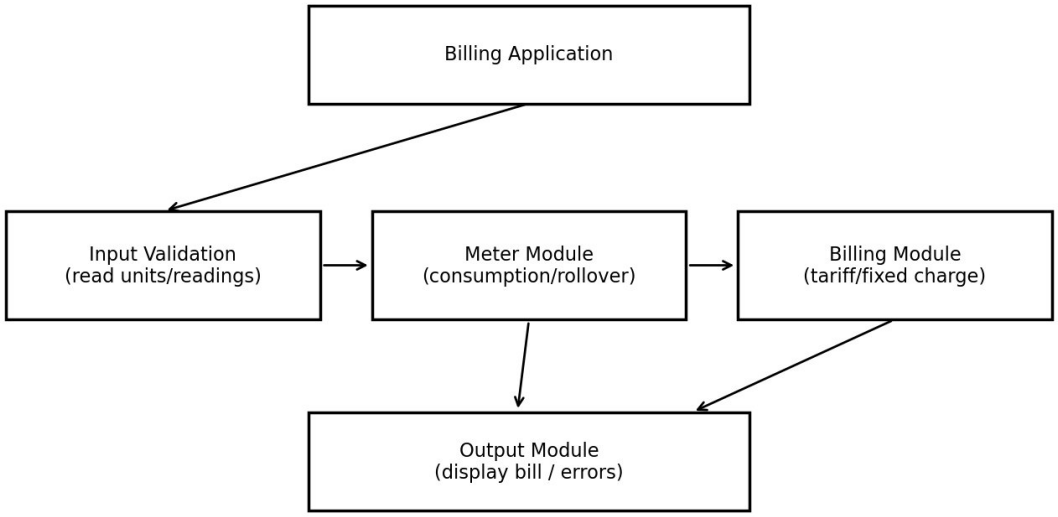


Figure 2. Simple module diagram showing a logical decomposition of the billing application. This is a module-level design diagram, not a claim that these are implemented as separate C classes.



6. Implementation Evidence: Tasks 01–12

Task 01: Problem analysis

analysis.txt records inputs, processing, outputs, alternative representation, and O(1) complexity.

```
Task 01 – Problem Analysis

Problem: Design a reliable electricity billing program for a rural electricity co-operative.

1. Inputs
- Previous meter reading.
- Current meter reading.
- Meter readings are stored in a 16-bit unsigned register ranging from 0 to 65535.
- The meter register can roll over from 65535 to 0.

2. Processing
- Calculate electricity consumption from the previous and current readings.
- Handle meter rollover correctly.
- Apply the billing rules:
  - Fixed charge: ₹50.
  - First 100 units: free.
  - Units 101–200: ₹2.25 per unit.
  - Units above 200: ₹4.50 per unit.
- Calculate the final bill accurately to the nearest paisa.
- Validate user input and handle invalid readings.

3. Outputs
- Electricity units consumed.
- Fixed charge.
- Energy charge.
- Total bill in rupees and paise.
```

```
- Error message for invalid input.

4. Alternative representation of meter readings

Meter readings can be represented as integers or strings.

- Integer representation:** Uses numeric values directly and supports arithmetic operations efficiently.
- String representation:** Stores digi
```

Task 02: Meter type and rollover

main.c defines meter_t as unsigned short and tests normal consumption 1200→1530 and rollover 65400→120.

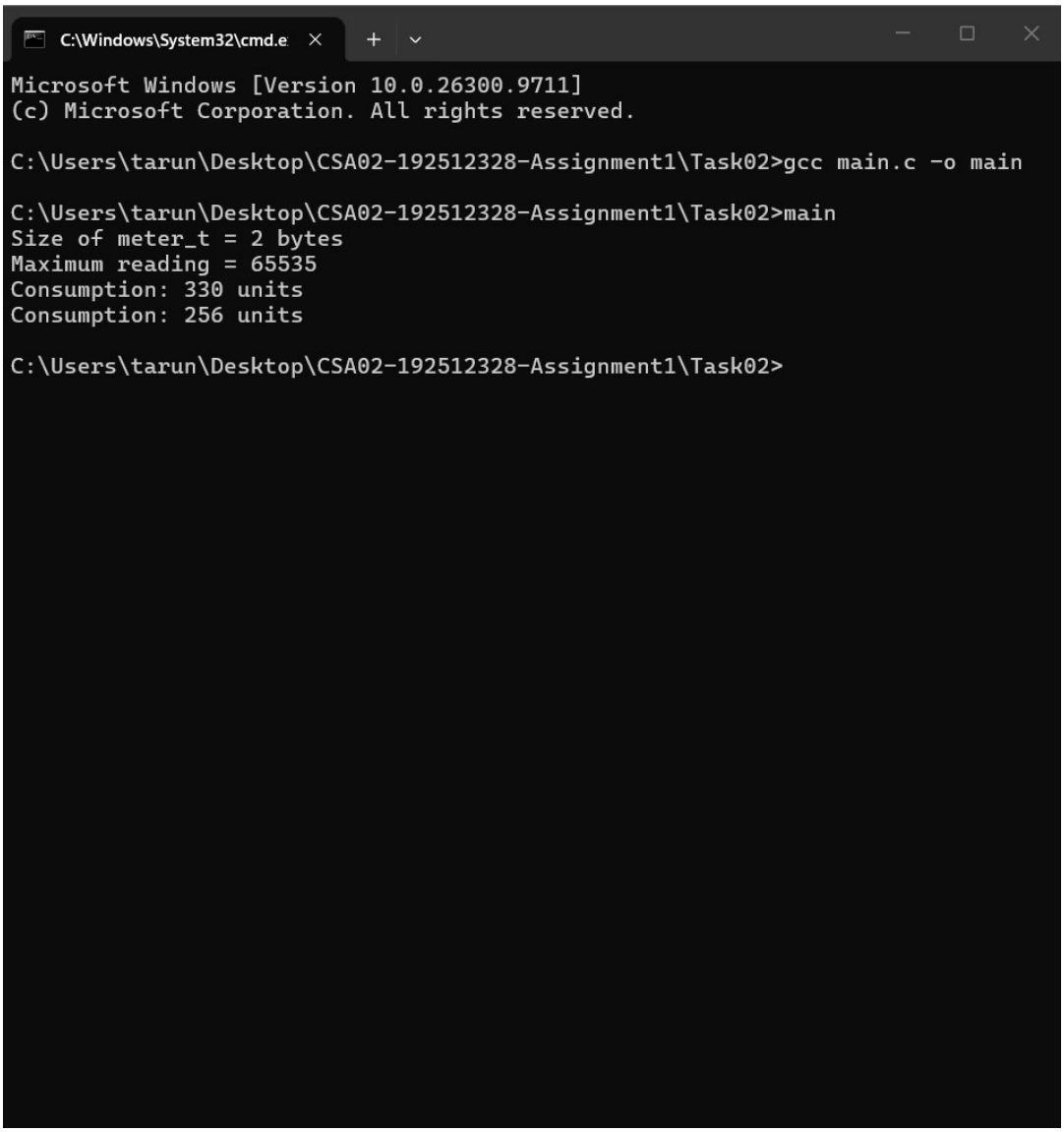
```
#include <stdio.h>
#include <limits.h>

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
   return (meter_t)(current - previous);
}

int main(void)
{
   printf("Size of meter_t = %zu bytes\n", sizeof(meter_t));
... (see full file in Task02 folder)
```

Evidence screenshot for Task 02:



Task 03: Tariff calculation with constants

main.c defines slab thresholds, rates in paise, fixed charge, and compute_bill().

```
#include <stdio.h>

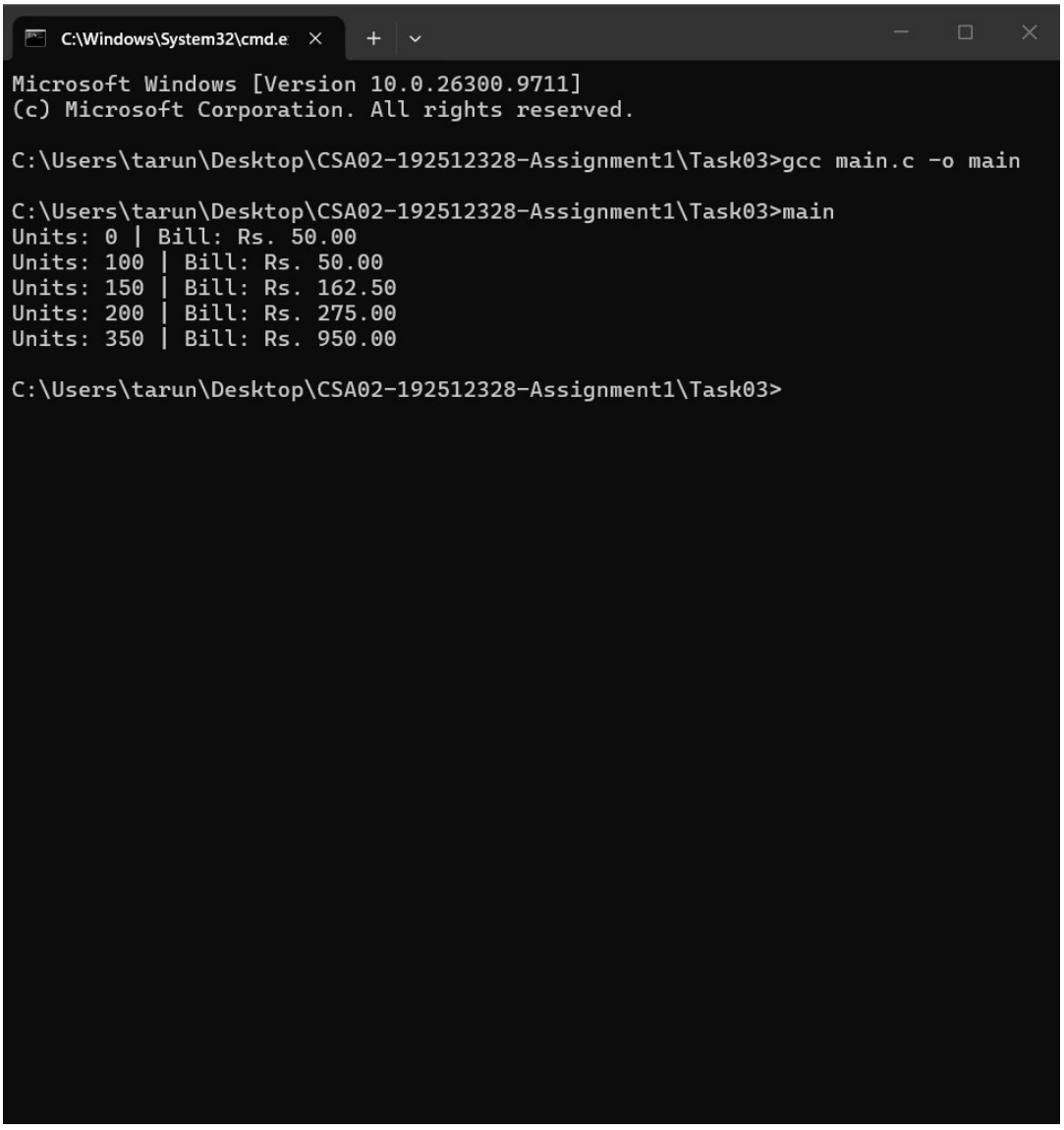
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200

#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450

#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units)
{
... (see full file in Task03 folder)
```

Evidence screenshot for Task 03:



Task 04: Macro safety

main.c contrasts SQUARE_BAD(x) with parenthesized SQUARE(x) and a static inline square() function.

```
#include <stdio.h>

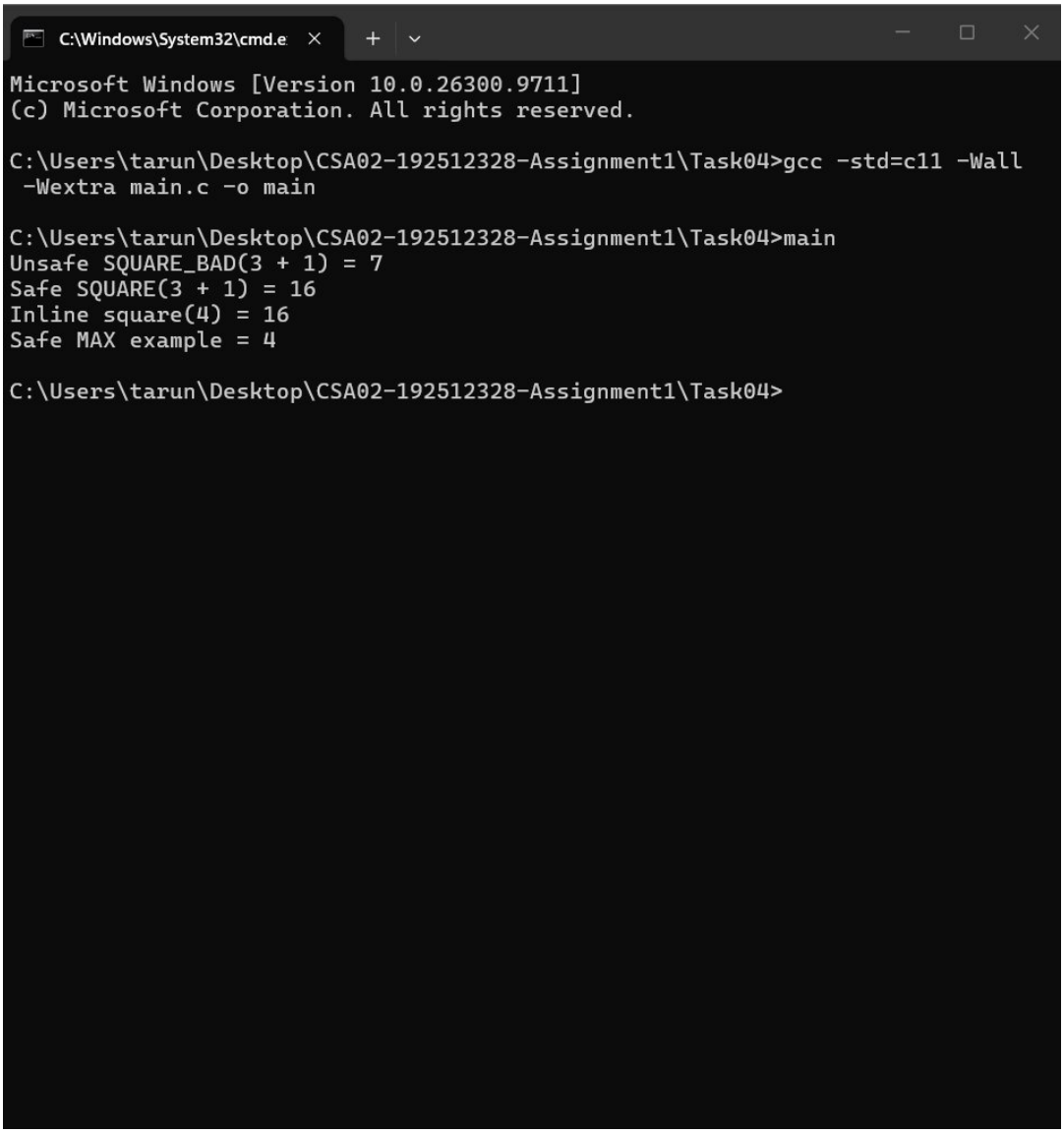
/* Unsafe macro */
#define SQUARE_BAD(x) x * x

/* Safer macro with parentheses */
#define SQUARE(x) ((x) * (x))

/* Safer alternative using an inline function */
static inline int square(int x)
{
    return x * x;
```

```
}
... (see full file in Task04 folder)
```

Evidence screenshot for Task 04:



Task 05: Include guard and conditional compilation

main.c demonstrates an include-guard pattern and optional DEBUG output.

```
#include <stdio.h>

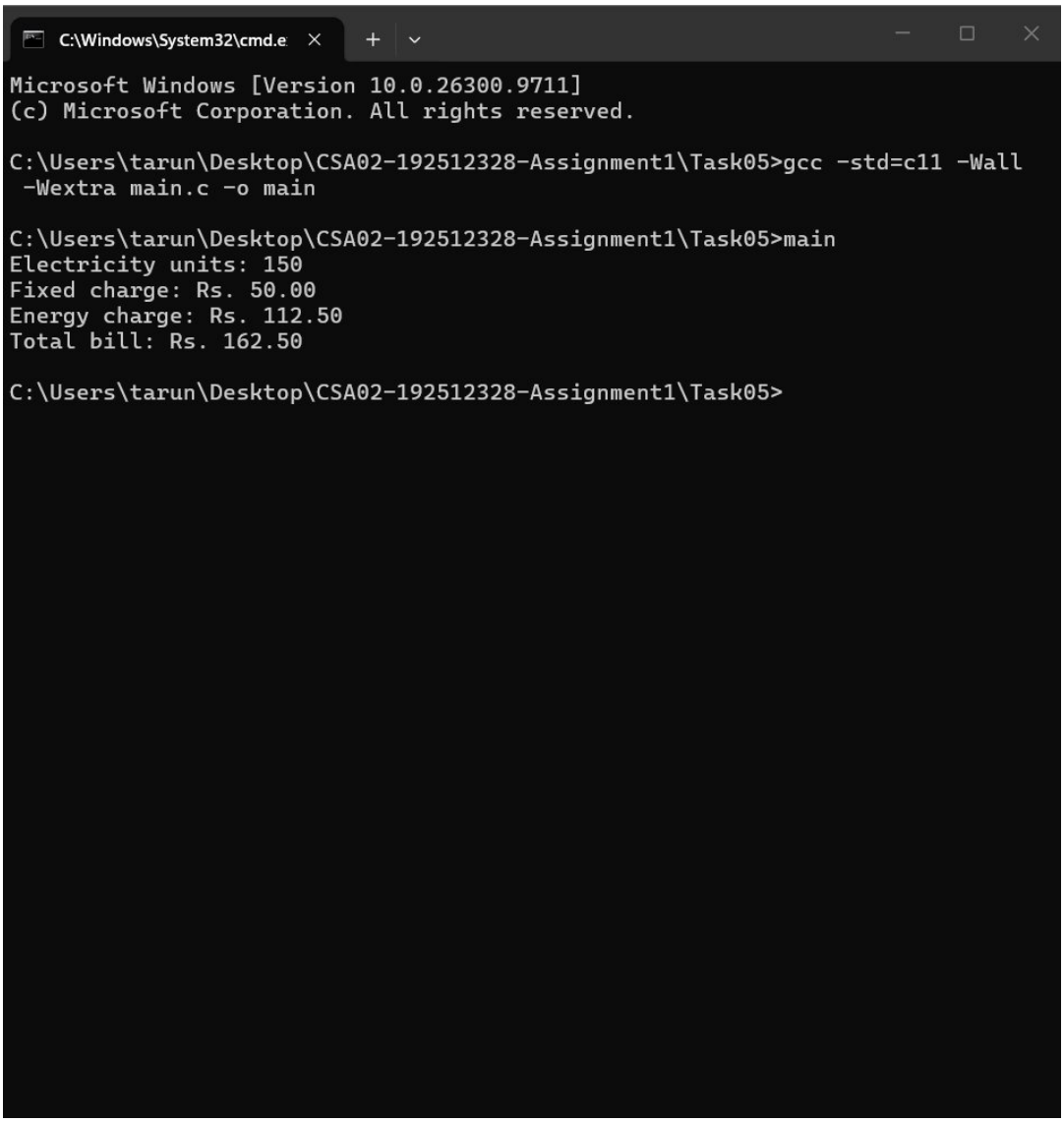
/* Include guard demonstration */
#ifndef BILLING_CONFIG_H
#define BILLING_CONFIG_H

#define FIXED_CHARGE 50
#define RATE_PER_UNIT 2.25

#endif
```

```
int main(void)
{
... (see full file in Task05 folder)
```

Evidence screenshot for Task 05:



Task 06: Input validation

main.c checks scanf() return value and rejects negative units.

```
#include <stdio.h>

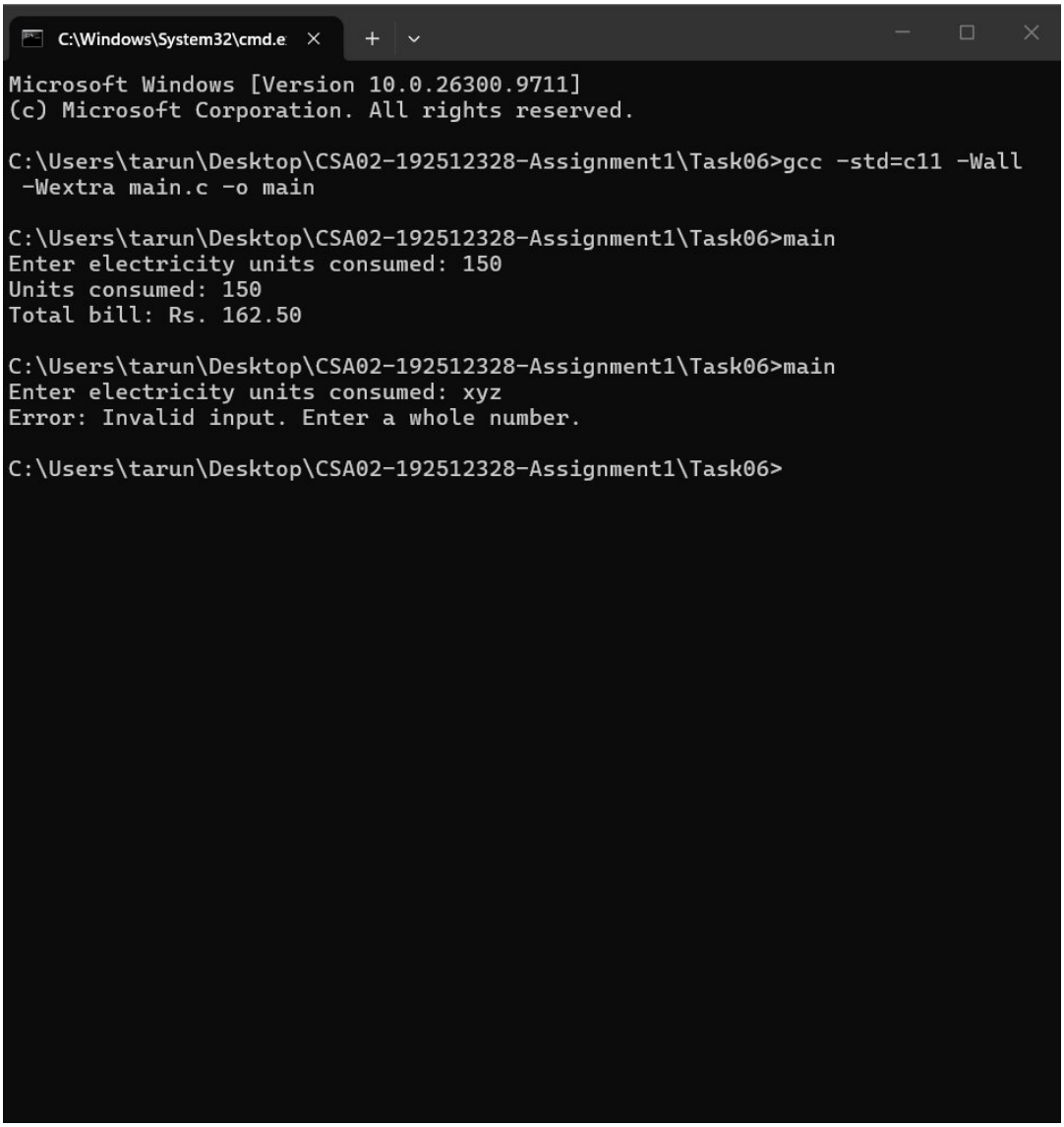
int main(void)
{
   int units;
   int result;

   printf("Enter electricity units consumed: ");
```

```
    result = scanf("%d", &units);

    if (result != 1) {
        printf("Error: Invalid input. Enter a whole number.\n");
... (see full file in Task06 folder)
```

Evidence screenshot for Task 06:



Task 07: Type conversion and precision

main.c demonstrates integer division, double conversion, truncation, integer paise, and double display.

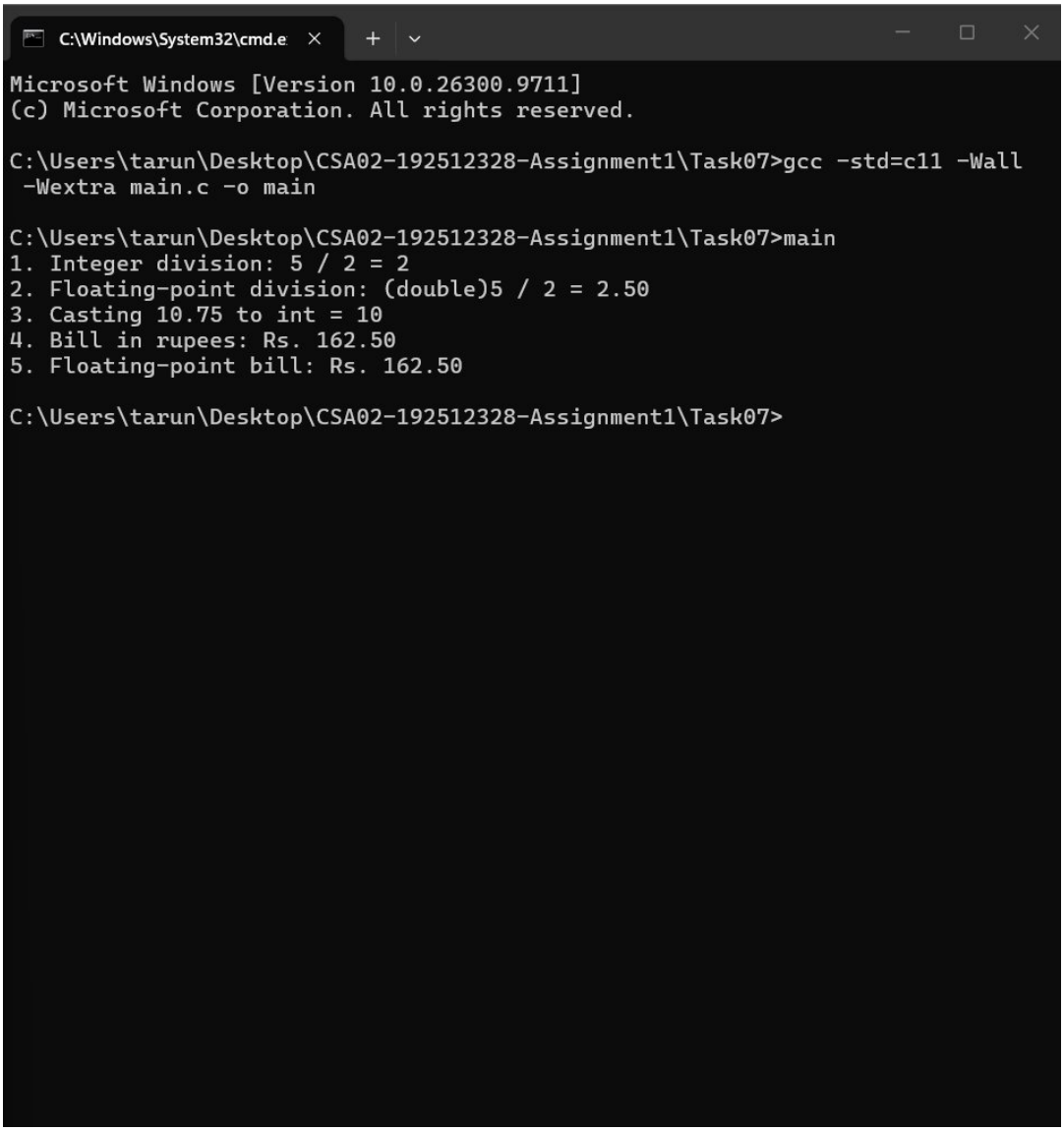
```
#include <stdio.h>

int main(void)
{
    int a = 5;
    int b = 2;
```

```
    /* Example 1: Integer division */
    printf("1. Integer division: 5 / 2 = %d\n", a / b);

    /* Example 2: Explicit type conversion */
    printf("2. Floating-point division: (double)5 / 2 = %.2f\n",
           (double)a / b);
... (see full file in Task07 folder)
```

Evidence screenshot for Task 07:



Task 08: Compilation stages

The folder includes main.c, main.i, main.s, main.o, main.exe, and output.png.

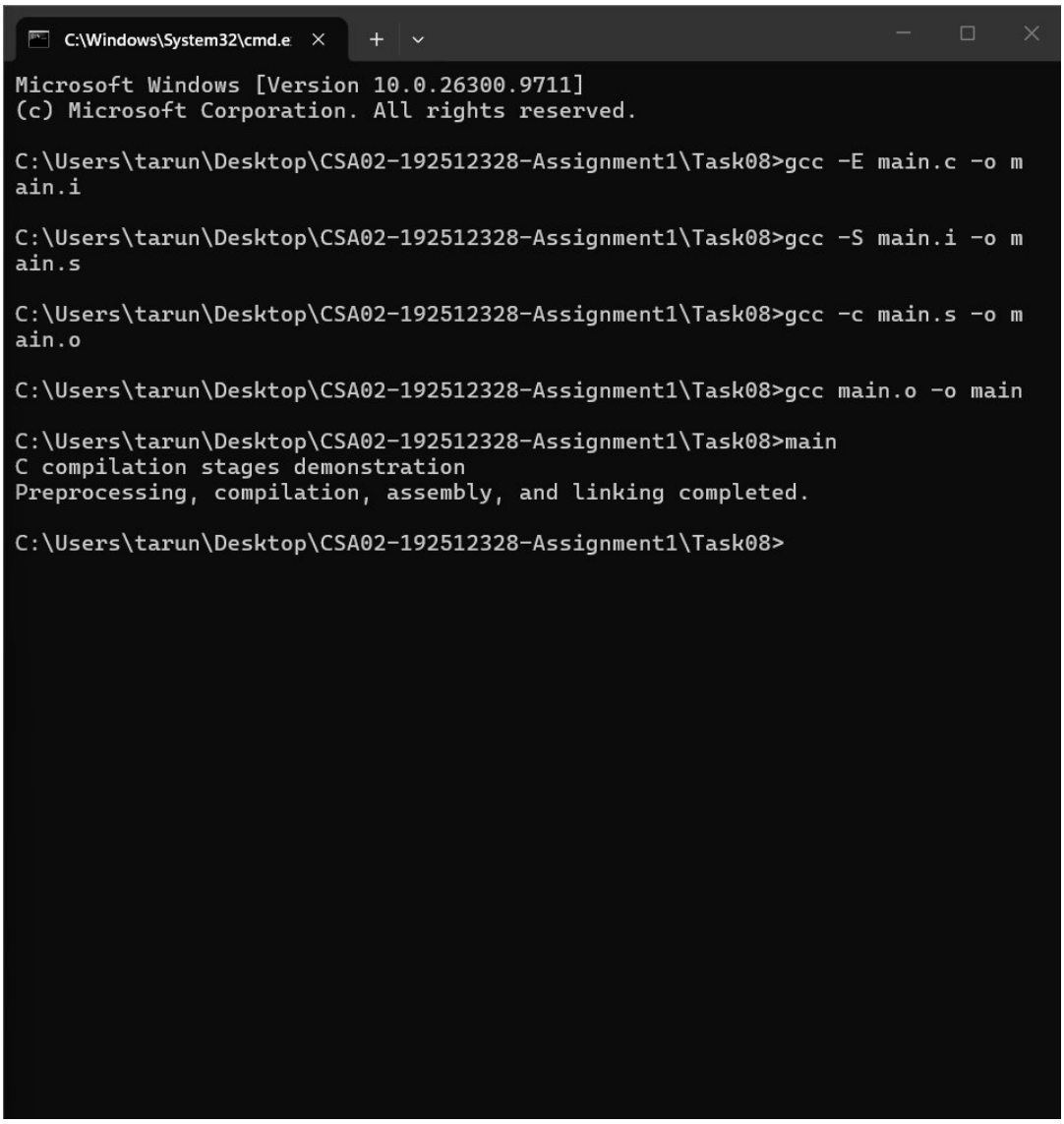
```
#include <stdio.h>

int main(void)
{
```

```
    printf("C compilation stages demonstration\n");
    printf("Preprocessing, compilation, assembly, and linking completed.\n");

    return 0;
}
```

Evidence screenshot for Task 08:



Task 09: Combined meter and billing tests

main.c combines rollover examples with tariff boundary cases.

```
#include <stdio.h>
#include <limits.h>

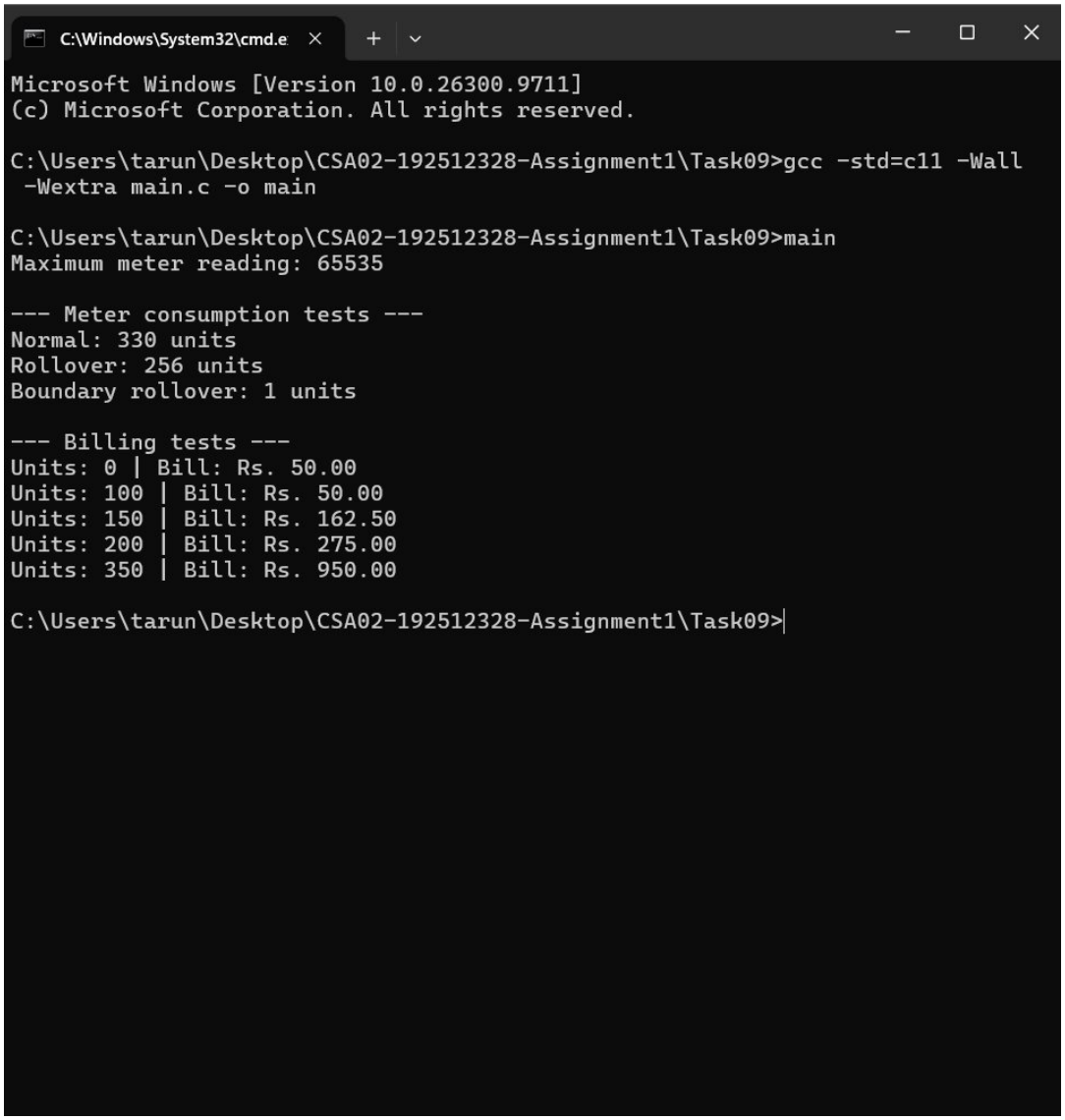
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE2_PAISE 225L
#define RATE3_PAISE 450L
```

```
#define FIXED_CHARGE_PAISE 5000L

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
... (see full file in Task09 folder)
```

Evidence screenshot for Task 09:



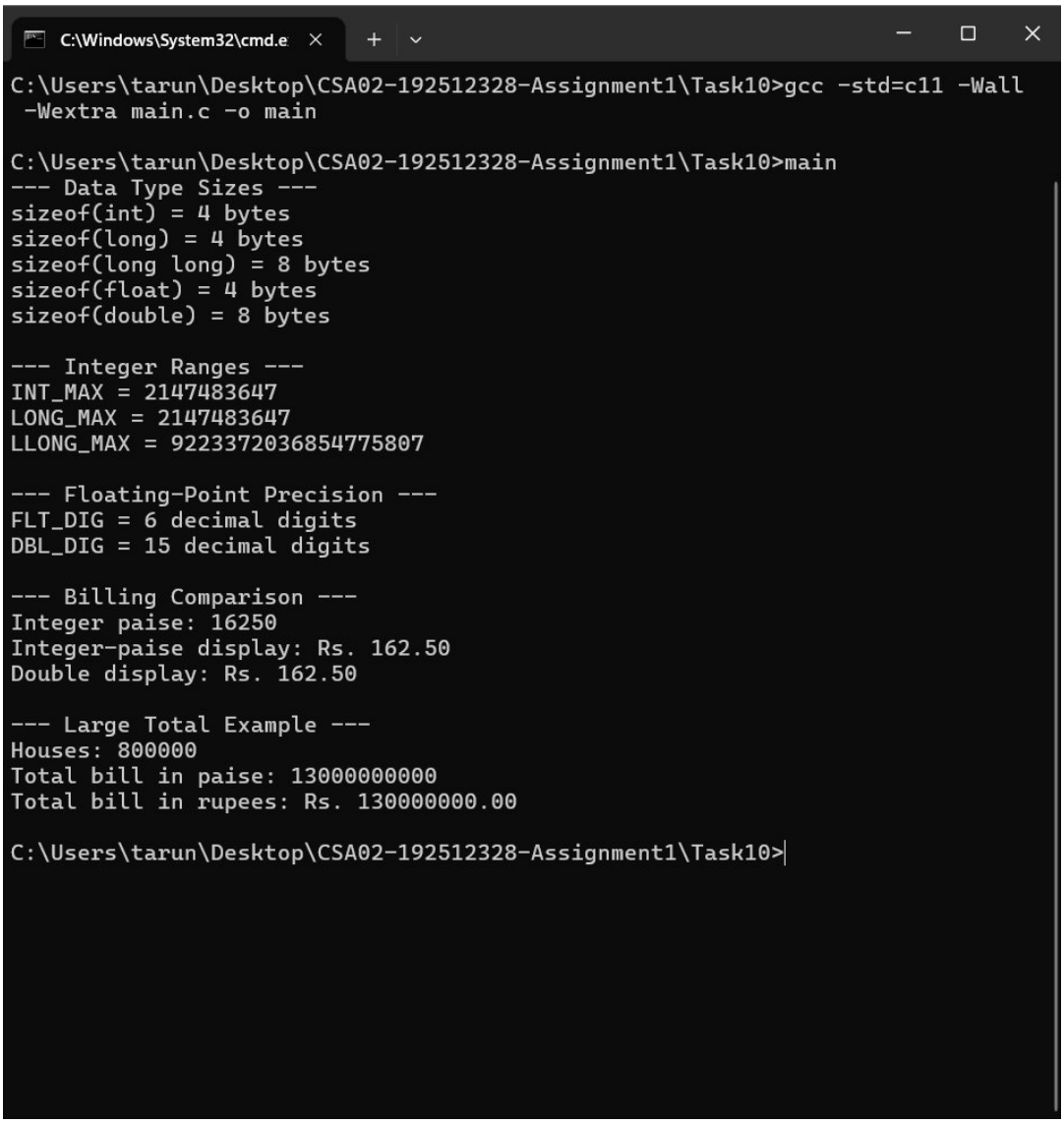
Task 10: Data types and large totals

main.c prints type sizes/ranges, precision indicators, and a large accumulated total.

```
#include <stdio.h>
#include <limits.h>
#include <float.h>
#include <stdint.h>
```

```
int main(void)
{
    printf("--- Data Type Sizes ---\n");
    printf("sizeof(int) = %zu bytes\n", sizeof(int));
    printf("sizeof(long) = %zu bytes\n", sizeof(long));
    printf("sizeof(long long) = %zu bytes\n", sizeof(long long));
    printf("sizeof(float) = %zu bytes\n", sizeof(float));
    printf("sizeof(double) = %zu bytes\n", sizeof(double));
... (see full file in Task10 folder)
```

Evidence screenshot for Task 10:



Task 11: Design recommendations

design_notes.txt discusses types, constants, integer paise, rollover, validation, testing, and improvements.

```
Task 11 – Final Design Recommendations

1. Data Types
```

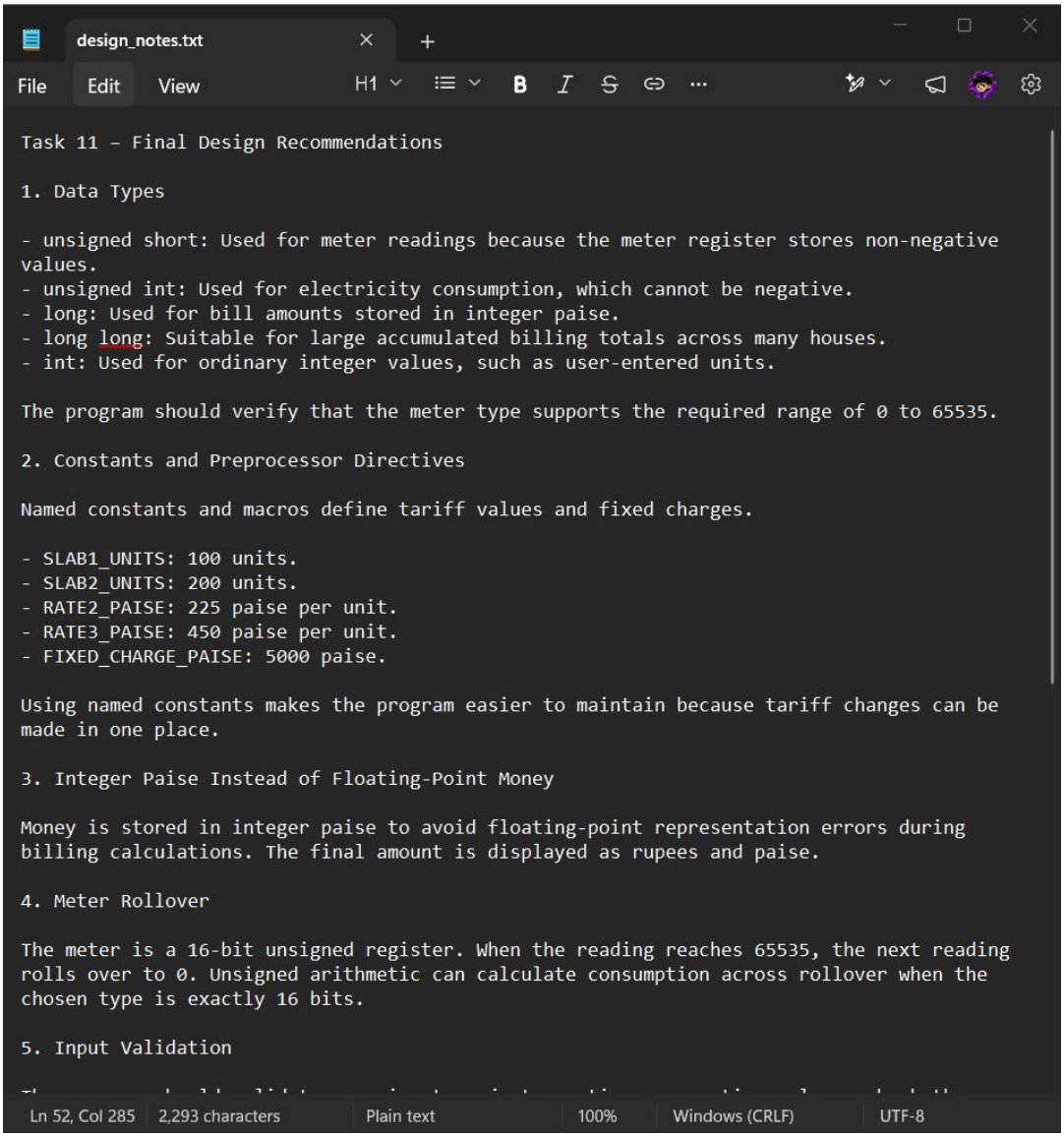
```
- unsigned short: Used for meter readings because the meter register stores non-negative values.
- unsigned int: Used for electricity consumption, which cannot be negative.
- long: Used for bill amounts stored in integer paise.
- long long: Suitable for large accumulated billing totals across many houses.
- int: Used for ordinary integer values, such as user-entered units.

The program should verify that the meter type supports the required range of 0 to 65535.

2. Constants and Preprocessor Directives

... (see full file in Task11 folder)
```

Evidence screenshot for Task 11:



Task 12: Smart meter design challenge

design_notes.txt describes 15-minute readings, peak tariff, runtime configuration, storage, validation, and tests.

```
Task 12 – Smart Meter Design Challenge
```

1. Problem Description

The smart meter system records electricity usage every 15 minutes. Each house produces 35,040 readings per year. The system must calculate electricity bills using different tariff rates depending on the time of day.

2. Time-of-Day Tariff

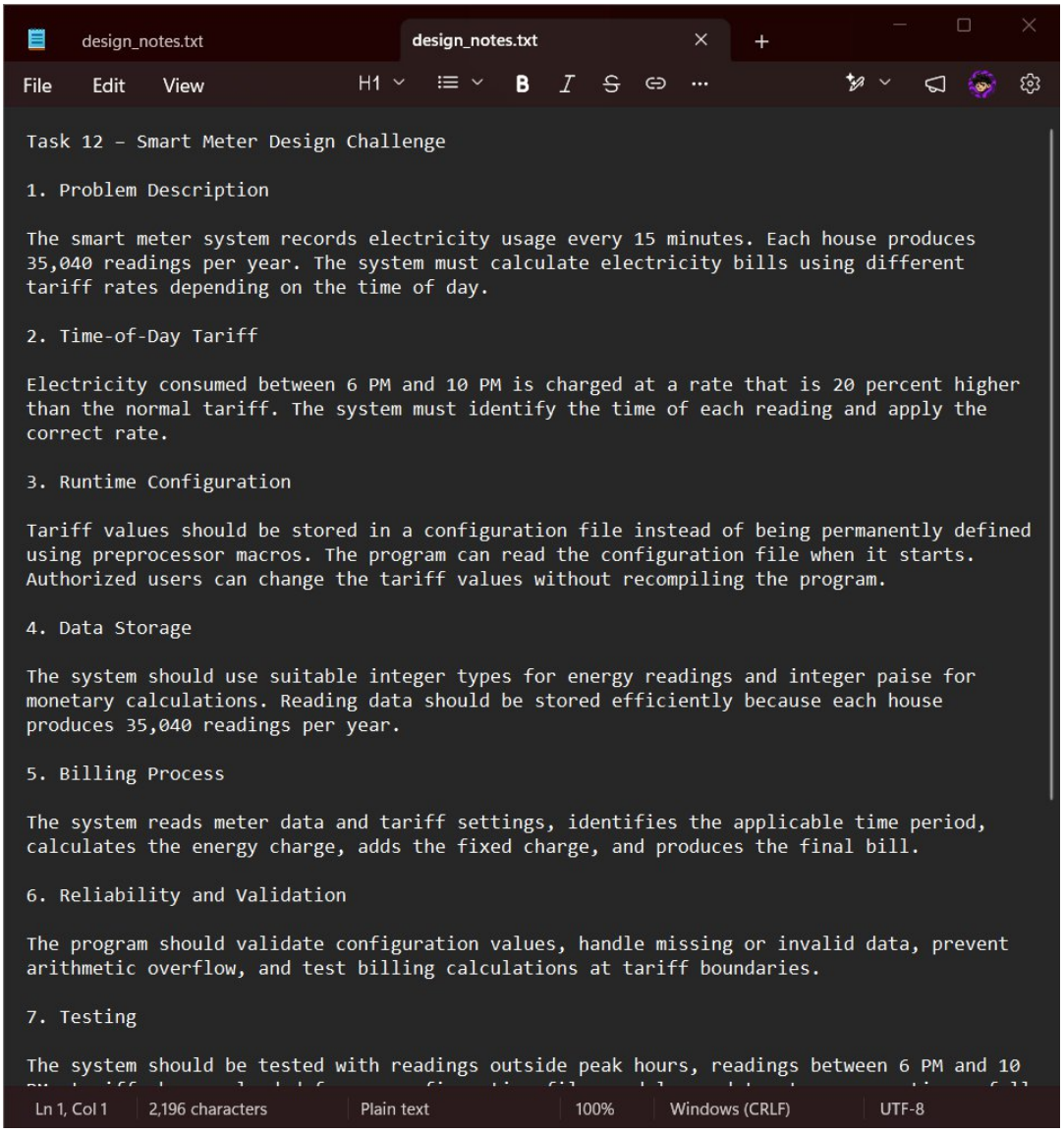
Electricity consumed between 6 PM and 10 PM is charged at a rate that is 20 percent higher than the normal tariff. The system must identify the time of each reading and apply the correct rate.

3. Runtime Configuration

Tariff values should be stored in a configuration file instead of being permanently defined using preprocessor macros. The program can read the configuration file when it starts. Authorized users can change the tariff values without recompiling the program.

... (see full file in Task12 folder)

Evidence screenshot for Task 12:



7. Test Plan and Results

The following expected results are calculated from the assignment tariff. “Expected” means formula-derived; the attached screenshots are the evidence files supplied in the project archive. Verify each actual run against these values before claiming a test passed.

Test	Input	Expected result
Normal meter use	Previous 1200; current 1530	330 units
Meter rollover	Previous 65400; current 120	256 units
Boundary rollover	Previous 65535; current 0	1 unit (16-bit register)
Zero units	0	₹50.00
First slab boundary	100	₹50.00
Second slab example	150	₹162.50
Second slab boundary	200	₹275.00
Third slab example	350	₹950.00
Negative units	-10	Reject with error
Non-numeric input	abc	Reject with error if tested by Task 06 program

7.1 Billing Calculation Workings

Units	Energy charge	Fixed charge	Total bill
0	₹0.00	₹50.00	₹50.00
100	₹0.00	₹50.00	₹50.00
150	50 × ₹2.25 = ₹112.50	₹50.00	₹162.50
200	100 × ₹2.25 = ₹225.00	₹50.00	₹275.00
350	₹225.00 + 150 × ₹4.50 = ₹900.00	₹50.00	₹950.00

Evidence note: the source archive includes output screenshots for Tasks 02–12. Task 01 is represented by analysis.txt. The ZIP does not include a separate invalid_input.png file, so do not claim a separate invalid-input screenshot unless you add one.

8. Compilation Stages

Stage	GCC command	Output
Preprocessing	gcc -E main.c -o main.i	Expanded source
Compilation to assembly	gcc -S main.i -o main.s	Assembly source
Assembly	gcc -c main.s -o main.o	Object file
Linking	gcc main.o -o main	Executable

The Task 08 folder in the uploaded archive contains main.i, main.s, main.o, main.exe, and output.png. This supports the compilation-stage demonstration.

9. Complexity Analysis

Operation / task	Time complexity	Auxiliary space	Reason
Task 01: analyze a single fixed-width reading	O(1)	O(1)	Fixed number of input fields and arithmetic operations.
Task 02 / Task 09: consumption calculation	O(1)	O(1)	Subtraction, comparison, and rollover arithmetic.
Task 03 / Task 06 / Task 09: one bill	O(1)	O(1)	Fixed number of slab comparisons and arithmetic operations.
Task 04: macro / inline demonstration	O(1)	O(1)	A fixed set of expressions is evaluated.
Task 05 / Task 07 / Task 10 demonstrations	O(1)	O(1)	Fixed number of demonstrations and printed values.
Task 08: compilation	Not expressed as O(1)	Depends on source/object size	Compiler complexity depends on program size and compiler implementation.
Process n readings once	O(n)	O(1) streaming auxiliary space	Each reading is processed a constant number of times.
Store n readings in memory	O(n) processing to ingest	O(n) storage	One record is retained per reading.
Smart meter annual readings per house	n = 35,040	O(n) if retained	15-minute intervals × 96 readings/day × 365 days.

For a streaming design, working memory can remain O(1) apart from the storage system, but retaining all annual records requires O(n) persistent storage.

10. Numeric Accuracy and Overflow

The program uses integer paise in the primary billing calculations: ₹50 = 5000 paise, ₹2.25 = 225 paise, and ₹4.50 = 450 paise. This avoids floating-point rounding during whole-paise calculations.

The C language does not guarantee that unsigned short is 16 bits on every platform. The meter implementation should verify the range and explicitly use a suitable fixed-width type or check USHRT_MAX == 65535. Arithmetic must also be checked for overflow for large user-entered unit counts and aggregated totals.

Task 10 demonstrates a large total: 800,000 houses × 16,250 paise = 13,000,000,000 paise = ₹130,000,000.00. This value fits in a sufficiently wide long long on common platforms, but range assumptions should be verified.

11. Smart Meter Design Challenge

Requirement	Design response
Reading interval	One reading every 15 minutes.
Annual count	96 readings/day × 365 days = 35,040 readings per house per year.
Peak window	6 PM–10 PM; proposed tariff is 20% above normal tariff.
Runtime tariffs	Read validated tariff values from a configuration file instead of compile-time-only macros.
Storage	Store meter identifier, timestamp, reading, and audit information.
Reliability	Validate missing/duplicate readings, tariff configuration, resets, and arithmetic limits.

The uploaded Task 12 is design notes rather than a complete implemented smart-meter system. Tariff treatment at interval boundaries and the precise conversion from meter readings to interval energy should be specified before production use.

12. Limitations and Future Improvements

The task programs are separate learning demonstrations, not a unified production application. The meter implementation assumes a 16-bit register and must verify that assumption. The basic billing program has no customer database, telemetry interface, or persistent billing ledger.

Future improvements include combining the demonstrations into a modular application, adding automated unit tests, strengthening input and overflow validation, separating declarations into header files, storing tariff configuration at runtime, and adding secure meter communication and audit logging.

13. Conclusion

This assignment demonstrates C programming techniques for electricity billing, including data types, integer-paise calculations, meter rollover, preprocessor directives, input validation, type conversion, compilation stages, testing, and complexity analysis. The included smart-meter proposal extends the basic scenario to time-of-day tariffs and 15-minute readings. The report distinguishes implemented task demonstrations from proposed future functionality.

14. References

• Assignment brief: Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative.

• GCC documentation: compiler options and preprocessing/compilation stages.

• C standard library documentation for input/output functions and numeric limits.

• Course materials and lecture notes supplied for the C Programming Assignment 1.

APPENDIX B. TASK-BY-TASK ALTERNATIVE CONCEPTS AND COMPLEXITY ANALYSIS

This appendix addresses the alternative-concept questions stated in the original assignment brief. Complexity is stated for the operation being compared.

O(1) means constant time or space for one fixed-size value.
O(d) means proportional to the number of digits.
O(L) means proportional to an input line length.
O(s) means proportional to the number of tariff slabs.
O(n) means proportional to the number of bills or readings.

Actual elapsed-time measurements must be collected on the student's own computer. No unrun timings are represented as measured results.

TASK 1 — INTEGER METER READINGS VERSUS TEXT STRINGS

Integer approach:
Read the meter value into an integer type and perform arithmetic directly.

Text approach:
Preserve the digits in a character array, so a value such as "01530" retains its leading zero. Validate every character, then convert to a numeric value before arithmetic.

Pseudocode for the text approach:

Read the meter reading as a string.
For each character in the reading:
If the character is not a digit, reject the input.
Convert the string to an integer.
If the value is outside 0–65535, reject the input.
Calculate consumption using unsigned meter arithmetic.

For d digits, validation and conversion take O(d) time, and the string requires O(d) space. A fixed-width integer reading uses O(1) time and O(1) space.

Both are constant with respect to the bounded five-digit meter domain, but O(d) describes text processing when digit count is considered variable.

For two readings, string validation and conversion take O(d1 + d2) time and O(d1 + d2) temporary space if both strings are retained. Integer arithmetic after conversion is O(1).

Text input is useful for preserving formatting and validating raw input. Integer storage is simpler and more efficient for arithmetic.

TASK 2 — INTEGER PAISE VERSUS DOUBLE RUPEES

The integer-paise approach stores an amount as an exact whole number of paise. For example, ₹162.50 becomes 16250 paise.

A double approach stores rupees using a binary floating-point approximation and rounds when converting to the final currency amount.

Both perform one addition in O(1) time. Integer paise typically uses O(1) space per value. Double also uses O(1) space per value, but its exact byte size is implementation-dependent and must be reported using sizeof.

For n bills, summing takes O(n) time and O(1) extra space if only a running total is kept.

Floating-point rounding error can accumulate with n and depends on values and summation order. A fixed error should not be claimed without measuring the actual inputs and platform.

Integer paise avoids binary fraction representation error for whole-paise values, provided the chosen integer type does not overflow.

Pseudocode:

Set total_paise = 0.
Set total_rupees = 0.0.

For each bill:
total_paise = total_paise + bill_paise.
total_rupees = total_rupees + bill_rupees.

Round total_rupees to paise.
Compare the rounded floating-point total with total_paise.

Conduct experiments using 4,000, 60,000, and 800,000 identical bill amounts. Accumulate one total using double and another using long long paise. Print the double total rounded to paise, the integer total, and their difference. Record the actual output from the GCC system.

TASK 3 — DEFINE VERSUS CONST VARIABLES AND ENUM

define:
A preprocessor macro replaces its name with the specified text before compilation. It does not create a variable or occupy storage by itself. It has no C type checking and can be less convenient to inspect in a debugger.

const variable:
A const variable has a declared type and scope. The compiler can perform type checking, and the variable may be visible in debugging information. It may occupy storage, although the compiler can optimize that storage away.

enum:
An enumeration defines named integer constants. It is suitable for related integral values, such as status codes or a set of named categories. It is not appropriate for fractional tariff values such as ₹2.25 unless those values are represented in a suitable integer unit such as paise.

Time complexity:
For a fixed number of tariff constants, the running-time difference is generally O(1) per bill. Changing the representation does not inherently change the asymptotic time complexity.

Space complexity:
Macros require no variable storage by themselves. const variables and enumeration types may be represented differently by the compiler; actual storage and executable size depend on implementation and optimization.

Recommendation:
Use clearly named integer-paise constants for tariff values. Select #define, const, or enum according to whether the value is a preprocessor replacement, a typed object, or an integral named constant.

TASK 4 — MACROS VERSUS STATIC INLINE FUNCTIONS

A macro is expanded by the preprocessor at each use. For example, an unsafe SQUARE(x) written as x*x can give an incorrect result when passed an expression such as a+1. Fully parenthesizing the macro prevents precedence errors. A macro argument can also be evaluated more than once, which can be unsafe with expressions such as i++.

A static inline function uses normal C function syntax and provides type checking. Its argument expressions are evaluated once when passed as arguments. The compiler may inline the function, but it is not guaranteed to do so.

Pseudocode for a safer function:

```
static inline int square(int x)
{
return x * x;
}
```

For a fixed arithmetic operation, both approaches have O(1) time complexity per use and O(1) auxiliary space.

If a macro is used k times, the expanded expression may be copied at each use, potentially increasing code size with k. An inline function may reduce duplicated code if the compiler emits a shared function body, or it may be expanded at each call if the compiler chooses to inline it.

The actual call overhead and executable size depend on optimization and compiler decisions. Measure both versions using the same compiler flags and benchmark.

TASK 5 — COMPILE-TIME DEBUGGING VERSUS A RUN-TIME DEBUG FLAG

Compile-time approach:
Use #ifdef DEBUG to include debugging statements only when compiled with the DEBUG macro enabled.

Run-time approach:
Use a variable such as debug and execute if (debug) printf(...) when debugging output is needed.

Pseudocode for run-time debugging:

If debug is true:
Print intermediate values.

The compile-time approach removes disabled debug statements during preprocessing. The run-time approach retains the conditional test and usually the debug code in the executable, even when debugging is disabled.

Time complexity:
For a fixed number of debug statements, both approaches remain O(1) per program operation. The run-time version adds a conditional test at each location where the check is performed. If debugging output is enabled, output time depends on the amount of data printed.

Space complexity:
The compile-time version can omit disabled debug code. The run-time version may retain the debug instructions and format strings. Actual executable size must be measured because compiler optimization affects the result.

TASK 6 — SCANF VERSUS FGETS WITH SSCANF OR STRTOL

The scanf approach reads formatted input directly. Its return value must be checked to determine whether the expected number of items was successfully converted.

The alternative approach reads a complete line with fgets(), then converts the contents using sscanf() or strtol(). strtol() allows the program to inspect where conversion stopped and detect range errors.

Pseudocode:

Read a line using fgets().
Attempt to convert the line into an integer.
If conversion fails, reject the input.
If extra non-whitespace characters remain, reject the input.
If the value is outside the permitted range, reject the input.
Otherwise, use the validated value.

For a line of length L, scanning and conversion take O(L) time. A fixed-size input buffer uses O(1) space relative to input length; a buffer that grows with the line length requires O(L) space.

Both methods can have O(L) processing time. fgets() followed by strtol() provides better control over complete-line validation and recovery from invalid input. scanf() can leave unconsumed characters in the input stream, so careless recovery may cause repeated failures.

TASK 7 — FLOAT CALCULATIONS VERSUS INTEGER PAISE

The float approach performs calculations in floating-point rupees and rounds only once at the end. The integer-paise approach performs whole-paise arithmetic throughout the calculation.

For each value, both approaches use O(1) time and O(1) space. Summing n bills takes O(n) time and O(1) extra space when only a running total is stored.

Floating-point calculations may introduce approximation error. Rounding only once at the end can reduce repeated rounding, but it does not eliminate floating-point representation error. The resulting total error depends on the values, operation order, data type, compiler and platform.

Integer-paise arithmetic is exact for whole-paise amounts as long as the integer type does not overflow. If a fractional charge must be converted into paise, the rounding rule must be defined explicitly.

Run the same set of bill amounts through both approaches for increasing values of n. Record the total from each approach and the difference in paise. Do not claim a measured error until the experiment has actually been run.

TASK 8 — SINGLE-FILE COMPILATION VERSUS MULTI-FILE COMPILATION

Single-file approach:
Compile and link the complete program from one C source file.

Multi-file approach:
Separate the program into modules such as input.c, tariff.c and report.c. Place shared declarations and function prototypes in a header file. Compile each source file into an object file, then link the object files together.

Example build process:

gcc -c input.c -o input.o
gcc -c tariff.c -o tariff.o
gcc -c report.c -o report.o
gcc input.o tariff.o report.o -o billing.exe

For a change to one source file, the single-file approach generally recompiles the whole translation unit. In a multi-file design, only the changed source file and files affected by relevant header changes need recompilation, followed by linking.

Compilation time depends on source size, dependencies, compiler options and the build system. It should not be labelled O(1) without specifying a model.

Object-file storage depends on the compiled code and data. A multi-file project produces multiple object files, and their combined size must be measured. Separate compilation can reduce rebuild work for large projects but introduces additional files and build management.

Record the rebuild times and object-file sizes on the same computer using the same compiler flags.

TASK 9 — FIXED IF STATEMENTS VERSUS ARRAYS AND A LOOP

The fixed approach uses a set of if statements to calculate charges for the known tariff slabs.

The alternative stores slab limits and rates in arrays and loops through the slabs to calculate the energy charge.

Pseudocode:

Set energy_charge = 0.
For each tariff slab:
Determine how many units belong to that slab.
Multiply those units by the slab rate.
Add the result to energy_charge.
Add the fixed charge.
Return the total bill.

For s tariff slabs, the loop approach takes O(s) time per bill. The arrays require O(s) storage. The fixed approach uses O(1) time and O(1) space when the number of slabs is fixed.

For the current three-slab tariff, both approaches have constant practical cost because s is fixed. An array-based design is easier to extend when tariff slabs change, while fixed if statements can be simpler for a small, permanently fixed tariff structure.

Test both approaches against the same boundary values and compare the outputs.

TASK 10 — DISTRICT TOTAL IN DOUBLE VERSUS LONG LONG

A district total can be accumulated in double or long long. The double approach has a wide numeric range but uses floating-point representation. The long long approach stores integer paise exactly while the total remains within its representable range.

Each addition takes O(1) time and each accumulator uses O(1) space. Summing n bills takes O(n) time and O(1) extra space.

An integer accumulator can overflow if the total exceeds its maximum value. A floating-point accumulator may lose precision when the total becomes too large or when repeated values cannot be represented exactly. The point at which precision is lost depends on the actual values and data type.

Use the same bill amounts at 4,000, 60,000 and 800,000 household scales. Compare the integer-paise total with the floating-point total rounded to paise. Also check whether int, long and long long can represent the maximum total on the target system.

Record actual totals, differences, type sizes and limits. Do not substitute theoretical values for measured output.

TASK 11 — COMPILE-TIME TARIFFS VERSUS A CONFIGURATION FILE

Compile-time approach:
Tariff values are defined in the source code or preprocessor macros. A tariff revision requires editing the source and recompiling and redistributing the program.

Configuration-file approach:
The program reads tariff values from a file when it starts, validates the values and stores them in memory for use during billing.

Pseudocode:

Open the tariff configuration file.
Read the t tariff values.
Validate the values and slab limits.
If any value is invalid, stop with an error.
Store the validated values.
Use the stored tariff values to calculate bills.

Loading t tariff values takes O(t) time and O(t) space if all values are retained. For each bill, the calculation then uses the stored values. A configuration file allows authorized tariff changes without recompiling, but requires file handling, validation, error reporting and protection against invalid settings.

The compile-time approach has no configuration-loading step, but changes require a rebuild and redistribution. For a tariff that changes periodically, runtime configuration is more flexible. Record actual startup time and configuration size if measurements are required.

TASK 12 — SMART-METER REDESIGN AND COMPLEXITY

The changed requirements introduce 15-minute readings, a time-of-day tariff with a 20% increase between 6 PM and 10 PM, and tariff values that can change without recompilation.

Each house generates:

96 readings per day × 365 days = 35,040 readings per year.

For n readings, processing each reading once takes O(n) time. A streaming approach can use O(1) working memory apart from storage, while retaining all readings requires O(n) storage.

The revised design should use sufficiently wide integer types for watt-hours and yearly totals, validate meter readings and timestamps, and store tariff settings in a validated runtime configuration file. Monetary calculations should use integer paise when the tariff rules can be represented in whole paise.

Pseudocode:

Load and validate tariff configuration.

For each timestamped meter reading:
Validate meter ID, timestamp, reading and configuration.
Calculate interval energy using the defined rollover/reset policy.
Split energy at 18:00 and 22:00 boundaries if an interval crosses them.
Apply the normal or 1.20-times peak rate to each portion.
Accumulate charges in integer paise with overflow checks.
Store the reading and billing audit record.

The existing Task 12 folder contains design notes rather than a complete smart-meter implementation. Therefore, peak-hour processing and runtime tariff loading must be labelled as proposed pseudocode unless working code and execution screenshots are added.

Before claiming validation, test normal and peak boundary intervals, missing or duplicate readings, meter reset or rollover, invalid configuration and maximum annual totals.

CONCLUSION

The alternative approaches differ mainly in validation, flexibility, accuracy, memory use and maintenance. Integer meter values and integer-paise billing provide straightforward arithmetic. Strings, floating-point totals, run-time debugging, configuration files and multi-file builds can be appropriate in specific circumstances, but each introduces trade-offs.

Big-O analysis describes how work and storage grow as the input grows. Actual elapsed times, numeric differences and executable sizes must be obtained from experiments on the target computer and recorded separately from theoretical analysis.